



GILLES MUSY
SMART CONTRACT SECURITY

SECURITY REVIEW

StVault

Collateralized borrowing vault · Demonstration engagement

StVault - Security Review

Gilles Musy · Demonstration engagement · June 2026

1. Disclaimer

This document is a **demonstration security review** performed on **StVault**, a deliberately vulnerable sample contract authored by Gilles Musy for portfolio purposes. It is **not** a real client engagement: the contract was never deployed, holds no user funds, and the vulnerabilities it contains were intentionally introduced to illustrate review methodology, severity classification and proof-of-concept construction. All findings, proofs of concept and remediation guidance below are genuine analysis of the contract as written.

A security review is a time-, resource- and expertise-bound effort to surface as many issues as possible. It cannot prove the absence of vulnerabilities. Subsequent reviews, a bug-bounty program and on-chain monitoring are recommended for any production deployment.

2. Risk Classification

Severity is derived from the impact of an issue and the likelihood of its occurrence.

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

IMPACT

- **High** - significant loss of protocol assets or significant harm to a group of users.
- **Medium** - moderate loss or moderate harm to a group of users.
- **Low** - minor loss or harm to a small group of users.

LIKELIHOOD

- **High** - practical under realistic on-chain conditions at low attacker cost.
- **Medium** - conditionally incentivized but still plausible.
- **Low** - requires unlikely assumptions or significant attacker stake.

3. About StVault

StVault is a collateralized borrowing vault. Users deposit a volatile collateral token and borrow a stable asset against it up to a fixed 75% loan-to-value ratio. Collateral is priced in USD through a Chainlink-style price feed. The contract exposes deposit, borrow, repay and collateral-withdrawal flows, with an owner-funded liquidity pool for the borrow asset.

4. Executive Summary

A time-boxed review of the `StVault` contract was performed by Gilles Musy. The review combined manual analysis with a Foundry test-driven proof-of-concept suite. **Three issues** were identified: one High, one Medium and one Low. Each is accompanied by a passing PoC that demonstrates and quantifies the impact.

Project	StVault (demonstration target)
Type	Collateralized borrowing vault
Language	Solidity ^0.8.20
Scope	src/StVault.sol
Review commit	6e0fc095b1a489fb09fe056969151c4c45771a93
Methodology	Manual review · Foundry PoC · forge lint
Timeline	June 2026

5. Findings

SEVERITY		COUNT
High		1
Medium		1
Low		1
Total		3

ID	TITLE	SEVERITY	STATUS
H-01	Collateral price feed consumed without staleness or sign validation	High	Open
M-01	Reentrancy in withdrawCollateral via collateral-token transfer hook	Medium	Open
L-01	Origination fee truncated by division before multiplication	Low	Open

[H-01] Collateral price feed consumed without staleness or sign validation

Impact: High · **Likelihood:** Medium → **High**

Description. `collateralValueUsd()` reads `latestRoundData()` and uses only the `answer` field. It never checks `updatedAt` against a freshness window, never requires `answer > 0`, and hardcodes the feed scale to `1e8` instead of reading `priceFeed.decimals()`.

```
// no updatedAt check, no answer > 0 check, scale hardcoded
function collateralValueUsd(address user) public view returns (uint256) {
    (, int256 answer, , , ) = priceFeed.latestRoundData();
    return (collateralOf[user] * uint256(answer)) / 1e8;
}
```

On an L2, a sequencer outage or a frozen / last-known feed value leaves the vault pricing collateral from data that no longer reflects the market. A borrower opens a position against a stale-but-favorable price; once the true price is reflected, the position is undercollateralized and the shortfall is borne by the protocol. A non-positive `answer` is likewise accepted, and a hardcoded scale silently misprices any feed not reporting 8 decimals.

Impact. In the PoC, a borrower deposits 10 collateral units and borrows 14,900 of the stable asset while the feed reports a 14-day-old price of \$2,000. Once the live price (\$100) is reflected, the position is underwater by **~13,974** units of the borrow asset - a direct, unrecoverable loss to the borrow-asset pool. The loss scales to the full pool for a sufficiently capitalized actor.

Proof of concept. `test_H01_StaleOracle_DrainsPool` (passing). The control test `test_Control_FreshOracle_BoundsBorrow` shows the identical borrow reverts under a fresh, correct price - isolating feed handling as the sole cause.

Recommendation. Validate every read: require `updatedAt` within a per-feed staleness window aligned to the feed heartbeat; require `answer > 0`; derive the scale from `priceFeed.decimals()`. On L2, gate operations on a Chainlink sequencer-uptime feed (using `startedAt` plus a grace period). Revert cleanly when any check fails.

[M-01] Reentrancy in withdrawCollateral via collateral-token transfer hook

Impact: High · **Likelihood:** Low → **Medium**

Description. `withdrawCollateral()` transfers collateral out *before* zeroing `collateralOf[msg.sender]`, and is the only state-changing user entry point not guarded by `nonReentrant`.

```
function withdrawCollateral() external {
    require(debtOf[msg.sender] == 0, "outstanding debt");
    uint256 amount = collateralOf[msg.sender];
    require(amount > 0, "no collateral");
    collateralToken.safeTransfer(msg.sender, amount); // external call first
    collateralOf[msg.sender] = 0;                    // state cleared after
}
```

If the collateral token invokes a recipient callback on transfer (ERC-777 / ERC-1363, or any token with transfer hooks), the caller re-enters `withdrawCollateral()` while the recorded balance is still non-zero, withdrawing it more than once.

Impact. PoC: the attacker deposits 10 collateral units, re-enters once, and exits with 20 - the second 10 belonging to another depositor. Vault collateral reaches zero while the victim's accounting still shows a 10-unit credit they can no longer redeem. Loss bound: the entire collateral pool.

Severity note. Set to Medium because exploitation requires a collateral asset with transfer callbacks. It rises to **High** if such assets (ERC-777 / ERC-1363 / hook tokens) are within the intended collateral scope.

Proof of concept. `test_M01_Reentrancy_DrainsCollateral` (passing).

Recommendation. Apply checks-effects-interactions - zero `collateralOf` before the external transfer - and add `nonReentrant` to `withdrawCollateral()` for defense in depth.

[L-01] Origination fee truncated by division before multiplication

Impact: Low · **Likelihood:** Medium → **Low**

Description. `borrow()` computes the origination fee dividing before multiplying:

```
uint256 fee = (amount / BPS_DENOM) * borrowFeeBps;
```

The intermediate quotient is truncated, so any `amount` not divisible by `BPS_DENOM` yields a fee below the intended `amount * borrowFeeBps / BPS_DENOM`.

Impact. The PoC borrows a non-round amount and is charged **49 wei** less than the correct fee (charged `1e16`, correct `1e16 + 49`). Per-transaction loss is bounded by `(BPS_DENOM-1) * borrowFeeBps / BPS_DENOM` and is economically negligible, but it is a systematic under-collection of protocol revenue.

Proof of concept. `test_L01_FeeRounding_UnderCharged` (passing).

Recommendation. Multiply before dividing, or use `Math.mulDiv(amount, borrowFeeBps, BPS_DENOM)` with an explicit rounding direction.

6. Reproduction

All findings are reproducible from the reviewed commit:

```
git checkout 6e0fc09
forge install foundry-rs/forge-std
forge install OpenZeppelin/openzeppelin-contracts@v5.1.0
forge test -vv
```

This demonstration review was prepared by Gilles Musy. The StVault contract is an intentionally vulnerable artifact and must not be deployed. © 2026 Gilles Musy.